

DMN Boundary Events Link Events Collections

Mgr. Marián Macík

April 2026

Principal Software Quality Engineer

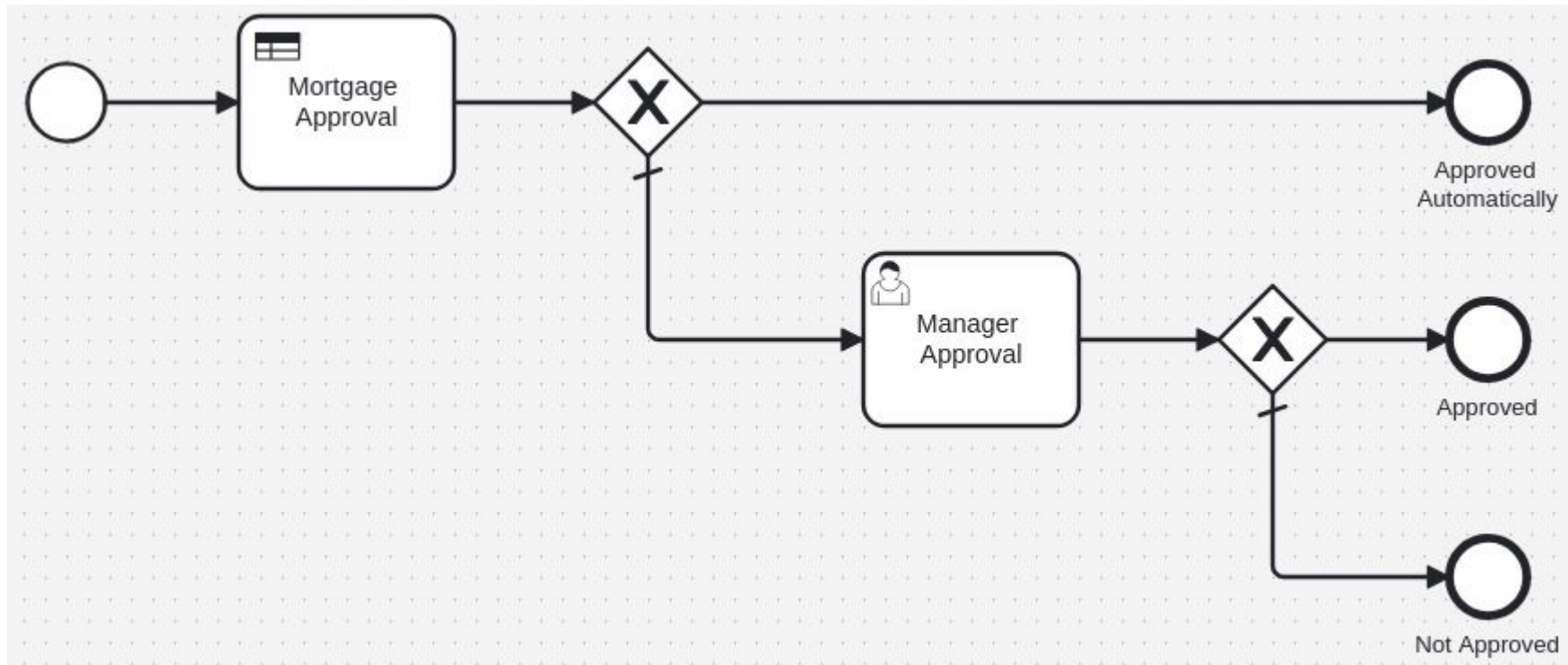
AGENDA

- ▶ DMN demo
- ▶ Boundary Error Events
- ▶ Link
- ▶ Collection parameters

DMN DEMO

The Process

- ▶ In a new project, create a new Process application and design the following process:



DMN DEMO

Process Start Form

- ▶ Create a new form and link it to the Start Node of the process
- ▶ Add these fields to the form:
 - **Price** of type **Number** with key **price**
 - **Age** of type **Number** with key **age**
- ▶ Set both of them as required, you can also add further validation, e.g. only positive values

Price*

Age*

DMN DEMO

Manager Approval Form

- ▶ Create a new form and link it to the Manager Approval user task
- ▶ Add these fields to the form:
 - **Price** of type **Number** with key **price**
 - **Age** of type **Number** with key **age**
 - **Approved** of type **Checkbox** with key **mortgageApproved**
- ▶ Set Price and Age as read only

Price

Age

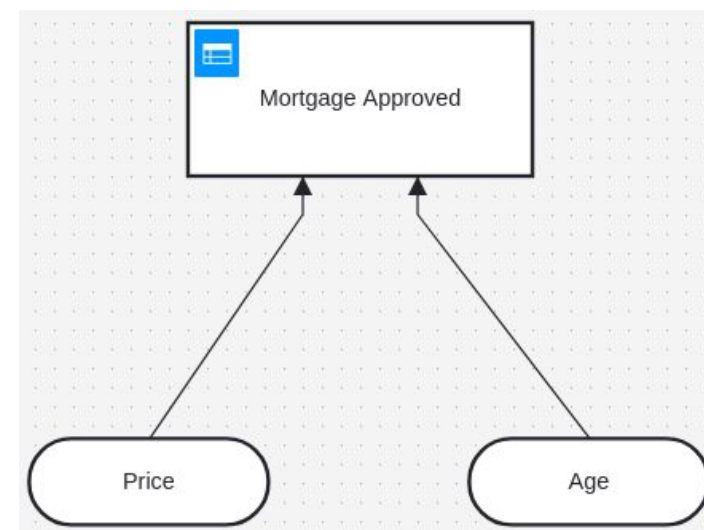
☐ Approved

DMN DEMO

DMN model

- ▶ Create a DMN diagram.
- ▶ Add 2 DMN Input Data nodes:
 - **Price** with ID **price**
 - **Age** with ID **age**
- ▶ Add **Mortgage Approved** DMN Decision node with ID **Decision_Mortgage** and click on a small table icon to open the decision table editor.
- ▶ Double-click on the Input and fill it according to the picture:
 - **price** of type **number**
 - **age** of type **number**
- ▶ Double-click on the Output and fill it according to the picture:
 - **Decision_Mortgage** of type **boolean**

Mortgage Approved		Hit policy: First		
	When	And	Then	Annotations
	price number	age number	+ Decision_Mortgage + boolean	
1	<5000	-	true	Everybody can have a loan of 5000
2	<10000	>25	true	Only age 26 and up can have up to 10 000 (exclusively)
3	-	-	false	
+	-	-		



DMN DEMO

Mortgage Approval Business Rule Task

- ▶ Open details of **Mortgage Approval** Business Rule Task in your process definition.
- ▶ Select **DMN decision** as **Implementation**.
- ▶ In the Called decision section, fill **Decision_Mortgage** as Decision ID and **mortgageApproved** as result variable
- ▶ Configure Inputs:

Inputs + 2 ▾

▾ age

Local variable name

age

Variable assignment value *fx*

= age

▾ price

Local variable name

price

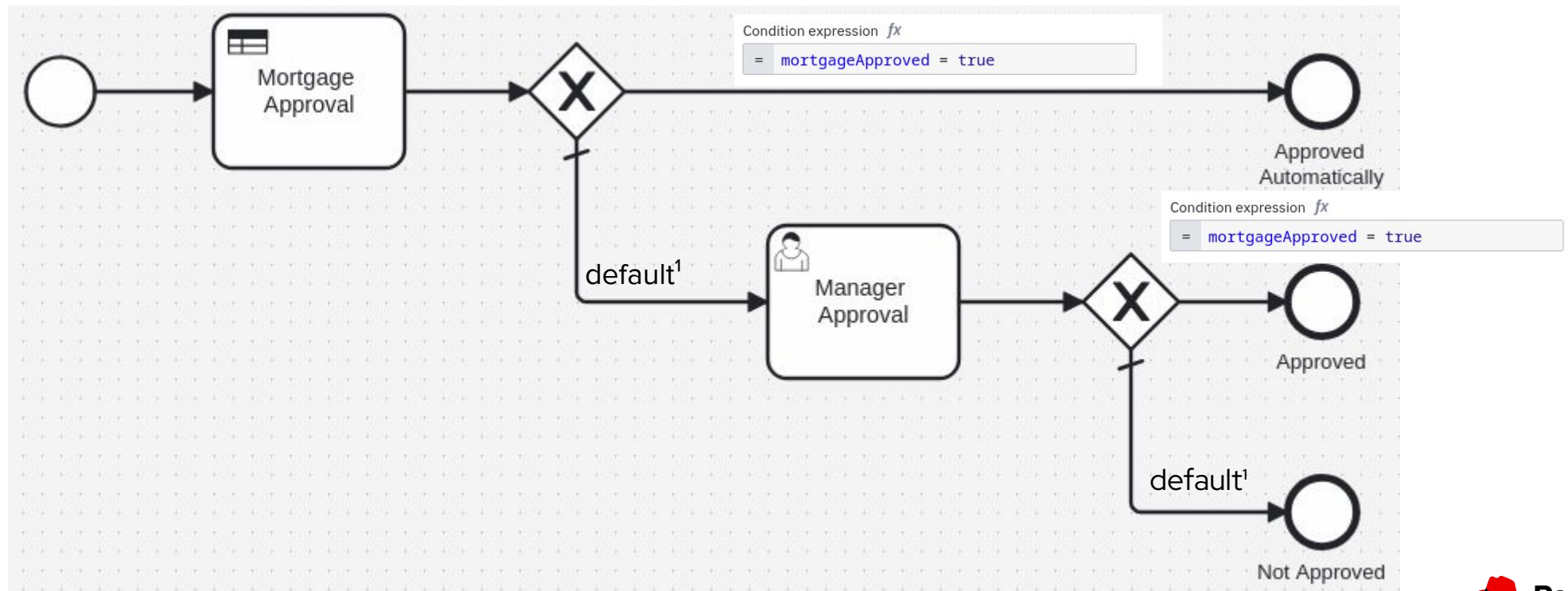
Variable assignment value *fx*

= price

DMN DEMO

Gateways

- Configure the gateways as on the picture:



¹Click on the sequence flow, click "Change element" and then select Default flow

DMN DEMO

Manager Approval Human Task

► Configure Inputs:

Inputs + 2 ▼

▼ price

Local variable name

price

Variable assignment value *fx*

= price

▼ age

Local variable name

age

Variable assignment value *fx*

= age

► Configure Outputs:

Outputs + 1 ▼

▼ mortgageApproved

Process variable name

mortgageApproved

Variable assignment value *fx*

= mortgageApproved

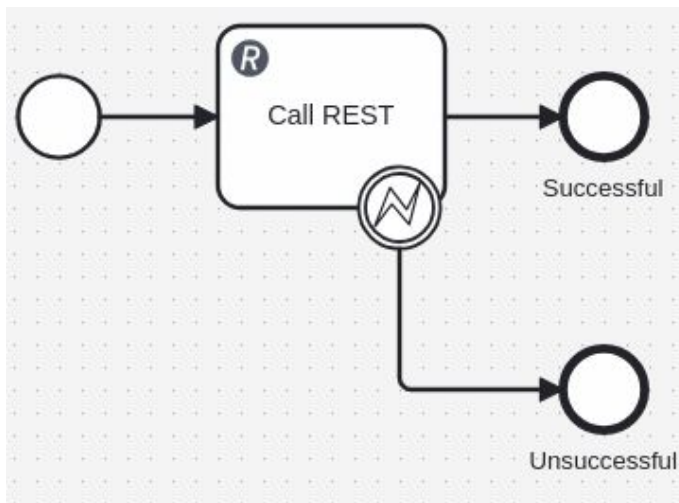
DMN DEMO

Run the Process

- ▶ Congratulations! You can now run your process and verify that it works.
- ▶ Because you created a Process application, you just need to deploy the main process and all other assets will be automatically deployed as well.

BOUNDARY ERROR EVENTS

- ▶ Handling of business exceptions explicitly by the process definition
- ▶ By default all errors on Connectors are technical errors
- ▶ To create a business error, additional configuration is needed



Error handling

Error expression *fx*

```
= if error.code = "404" then
    bpmnError("404", "Got 404 Not Found")
else // for other codes, process normally
    null
```

Expression to handle errors. Details in the [documentation](#).

Call REST configuration

- ▶ More information in [documentation](#)

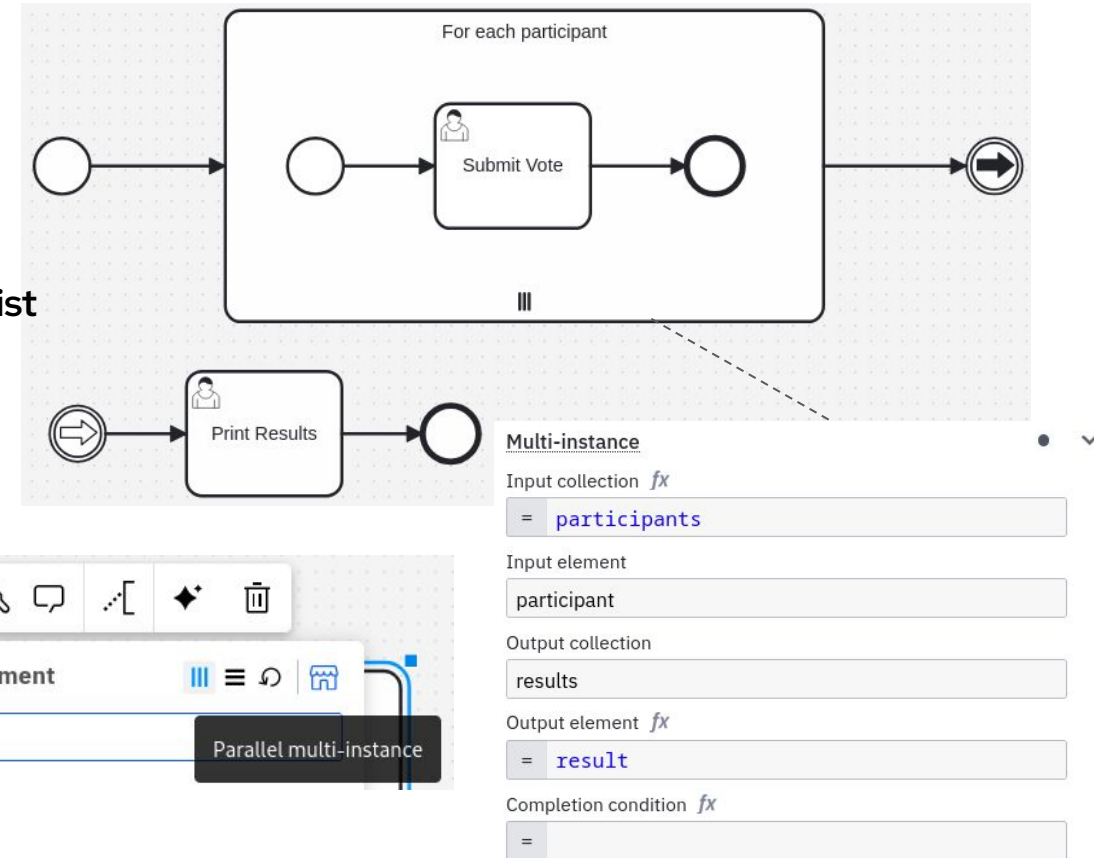
POLL EXAMPLE - LINK EVENT AND COLLECTIONS

- ▶ Create process definition called **Voting**, make sure to specify Embedded subprocess as Parallel multi-instance
- ▶ Assign a start form to it with fields:
 - **Participants** of type **Dynamic list** with path **participants**
 - **Name** of type **Text field** with key **name** inside that **Dynamic list**

Participants

Name

Repeatable



POLL EXAMPLE - LINK EVENT AND COLLECTIONS

- ▶ Create **VotingForm** and assign it to **Submit Vote** user task with fields:
 - **Text view** with text **# Which is your favourite browser {{participant.name}}?**
 - **Vote** of type **Radio Group** with key **result.vote**
 - Mark it as required
- ▶ Configure Inputs/Outputs of **Submit Vote** user task
- ▶ Create **Print Results** form and assign it to **Print Results** user task with fields:
 - **Results** of type **Dynamic list** with path **results**
 - **Vote** of type **Text field** with key **vote** inside that **Dynamic list**



Results

Vote

Repeatable

ABC Text view is templated

Vote*

- ☐ Firefox
- ☐ Chrome
- ☐ Edge
- ☐ Safari

Inputs + 1

participant

Local variable name

participant

Variable assignment value *fx*

= participant

Outputs + 1

result

Process variable name

result

Variable assignment value *fx*

= result

POLL EXAMPLE - LINK EVENT AND COLLECTIONS

- ▶ Configure Inputs of **Print Results** user task



Inputs + 1 ▼

▼ results

Local variable name

results

Variable assignment value *fx*

= results

- ▶ Link events - set link name, works like signals within a single process definition
- ▶ Deploy and start a process. Provide names of participants. After that, there will be **Submit Vote** user task created for each participant to cast a vote. After all votes are cast, process continues through link event and **Print Results** user task is created which displays each vote.

Thank you!



linkedin.com/company/red-hat



youtube.com/user/RedHatVideos



facebook.com/redhatinc



twitter.com/RedHat